# Lab: Advanced AI Agent Concepts

This lab teaches five core ideas behind learning agents by having you **build the key logic yourself**, not just run finished code. For each concept you get a working harness (the setup, the examples, the visualization). The important part, the actual learning rule, is left blank for you to write.

## How to use this notebook

- Cells marked `# TODO` are yours to complete. They will raise `NotImplementedError` until you finish them.
- Run cells top to bottom with **Shift + Enter**. Later parts reuse earlier code, so order matters.
- After each part there is an **Experiment** cell (change something and rerun) and a **Reflection** cell (answer in your own words, based on what you saw).
- Do not use a text AI to write your code or your answers. Your explanations must match the output your own code produced.

```
# Cell 1
# Setup (ready to run)
import random
print("Ready.")
```

```
Ready.
```

# Part 1 — Deep Q-Network (DQN)

A DQN learns the **value** of taking an action in a state. Here we store those values in a Q-table (a dictionary mapping `(state, action)` to a number). After each experience, we nudge the stored value toward what we actually observed.

**The Q-learning update formula:**

```
new_value = old_value + learning_rate * (reward + discount_factor * next_max - old_value)
```

where `next_max` is the best Q-value available from the next state.

**Your task (Cell below):** implement the `update` method using that formula.

```
# Cell 2
class SimpleDQN:
    def __init__(self):
        self.q_table = {}            # (state, action) -> value
        self.learning_rate = 0.1
        self.discount_factor = 0.9

    def get_q_value(self, state, action):
        # Ready to use. Returns 0.0 for unseen pairs.
        return self.q_table.get((state, action), 0.0)

    def update(self, state, action, reward, next_state):
        # TODO: implement the Q-learning update.
        # 1. old_value = current Q-value for (state, action)
        old_value = self.get_q_value(state, action)
        # 2. next_max  = the highest Q-value over actions ['move', 'stop'] in next_state
        possible_actions = ['move', 'stop']
        next_max = max([self.get_q_value(next_state, a) for a in possible_actions])

        # 3. new_value = old_value + learning_rate * (reward + discount_factor * next_max - old_value)
        new_value = old_value + self.learning_rate * (reward + self.discount_factor * next_max - old_value)

        # 4. store new_value back into self.q_table[(state, action)]
        self.q_table[(state, action)] = new_value
```

```
# Cell 3
# Run this to test your update (provided)
agent = SimpleDQN()
agent.update("obstacle_ahead", "stop", reward=10, next_state="clear_path")
print("After 1 update:", agent.get_q_value("obstacle_ahead", "stop"))

# Apply the SAME experience four more times and watch the value climb
for _ in range(4):
    agent.update("obstacle_ahead", "stop", reward=10, next_state="clear_path")
    print("Q-value:", round(agent.get_q_value("obstacle_ahead", "stop"), 4))
```

```
After 1 update: 1.0
Q-value: 1.9
Q-value: 2.71
Q-value: 3.439
Q-value: 4.0951
```

```
# EXPERIMENT (TODO)
# Make a new agent, set its learning_rate to 0.5, and repeat the same five updates.
# Compare how fast the value rises against the 0.1 agent above.

# TODO: write your experiment here.
```

```
# Cell 4 — New agent experiment
# Create a new agent with a higher learning rate
agent2 = SimpleDQN()
agent2.learning_rate = 0.5

print("Testing agent with learning_rate = 0.5")

# Apply the same experience five times
for _ in range(5):
    agent2.update("obstacle_ahead", "stop", reward=10, next_state="clear_path")
    print("Q-value:", round(agent2.get_q_value("obstacle_ahead", "stop"), 4))
```

```
Testing agent with learning_rate = 0.5
Q-value: 5.0
Q-value: 7.5
Q-value: 8.75
Q-value: 9.375
Q-value: 9.6875
```

```
# Cell 5 — My New agent experiment
# Create a new agent with a higher learning rate
agent2 = SimpleDQN()
agent2.learning_rate = .8

print("Testing agent with learning_rate = 0.5")

# Apply the same experience five times
for _ in range(5):
    agent2.update("obstacle_ahead", "stop", reward=10, next_state="clear_path")
    print("Q-value:", round(agent2.get_q_value("obstacle_ahead", "stop"), 4))
```

```
Testing agent with learning_rate = 0.5
Q-value: 8.0
Q-value: 9.6
Q-value: 9.92
```

```
Q-value: 9.984
Q-value: 9.9968
```

```
# Cell 6 — Another New agent experiment
# Create a new agent with a lower learning rate
agent2 = SimpleDQN()
agent2.learning_rate = .25

print("Testing agent with learning_rate = 0.5")

# Apply the same experience five times
for _ in range(5):
    agent2.update("obstacle_ahead", "stop", reward=10, next_state="clear_path")
    print("Q-value:", round(agent2.get_q_value("obstacle_ahead", "stop"), 4))
```

```
Testing agent with learning_rate = 0.5
Q-value: 2.5
Q-value: 4.375
Q-value: 5.7812
Q-value: 6.8359
Q-value: 7.627
```

**Reflection 1.** In two to three sentences, explain why the Q-value rose with each repeat, and what changing the learning rate did to how fast it rose.

_With lower learning rates, the Q-value increased slowly toward the long term expected reward and it is more stable. With a higher learning rate, each step is so large that the Q-value can quickly jump past the expected reward because the updates are just too large. Higher learning rates risk overshooting the expected reward and then bouncing back and forth.

## Part 2 — Policy Gradient

A policy-gradient agent does not store values. It stores a **probability** for each action and shifts those probabilities toward actions that earned reward.

**Your task:** implement `update_policy` so that when an action earns positive reward, its probability goes up, and then all probabilities are renormalized to sum to 1.

```
# Cell 7
class SimplePolicyAgent:
    def __init__(self):
```

```
        self.action_probabilities = {'move': 0.5, 'stop': 0.5}
        self.learning_rate = 0.1

    def choose_action(self):
        # Ready to use. Samples an action using the current probabilities.
        return random.choices(
            list(self.action_probabilities.keys()),
            list(self.action_probabilities.values())
        )[0]

    def update_policy(self, action, reward):
        # TODO:
        if reward > 0:

        #   1. add self.learning_rate to the probability of `action`
          self.action_probabilities[action] += self.learning_rate
        #   2. renormalize: divide every probability by the new total so they sum to 1
          total = sum(self.action_probabilities.values())
          for action in self.action_probabilities:
            self.action_probabilities[action] /= total
```

```
# Cell 8
# Run this to test (provided). Reward 'move', punish 'stop'.
policy_agent = SimplePolicyAgent()
for _ in range(8):
    a = policy_agent.choose_action()
    reward = 1 if a == 'move' else -1
    policy_agent.update_policy(a, reward)
    print({k: round(v, 3) for k, v in policy_agent.action_probabilities.items()})
```

```
{'move': 0.545, 'stop': 0.455}
{'move': 0.545, 'stop': 0.455}
{'move': 0.587, 'stop': 0.413}
{'move': 0.624, 'stop': 0.376}
{'move': 0.658, 'stop': 0.342}
{'move': 0.69, 'stop': 0.31}
{'move': 0.718, 'stop': 0.282}
{'move': 0.743, 'stop': 0.257}
```

```
# Cell 9
# EXPERIMENT (TODO)
# Copy the loop above but FLIP the reward rule so 'stop' is rewarded instead.
# Run it and watch which way the probabilities drift.
```

```
# TODO: write your experiment here.

# Run this to test (provided). Reward 'move', punish 'stop'.
policy_agent = SimplePolicyAgent()
for _ in range(8):
    a = policy_agent.choose_action()
    reward = -1 if a == 'move' else 1
    policy_agent.update_policy(a, reward)
    print({k: round(v, 3) for k, v in policy_agent.action_probabilities.items()})
```

```
{'move': 0.455, 'stop': 0.545}
{'move': 0.413, 'stop': 0.587}
{'move': 0.376, 'stop': 0.624}
{'move': 0.376, 'stop': 0.624}
{'move': 0.376, 'stop': 0.624}
{'move': 0.342, 'stop': 0.658}
{'move': 0.31, 'stop': 0.69}
{'move': 0.282, 'stop': 0.718}
```

```
# Cell 9.1
# BONUS: Add a third action 'jump' and test how probabilities drift

class ExtendedPolicyAgent(SimplePolicyAgent):
    def __init__(self):
        # Start with equal probabilities for three actions
        self.action_probabilities = {'move': 1/3, 'stop': 1/3, 'jump': 1/3}
        self.learning_rate = 0.1

# Test run: reward 'jump', punish 'move', neutral 'stop'
policy_agent = ExtendedPolicyAgent()
for _ in range(10):
    a = policy_agent.choose_action()
    if a == 'jump':
        reward = 1
    elif a == 'move':
        reward = -1
    else:  # 'stop'
        reward = 0
    policy_agent.update_policy(a, reward)
    print({k: round(v, 3) for k, v in policy_agent.action_probabilities.items()})
```

```
{'move': 0.333, 'stop': 0.333, 'jump': 0.333}
{'move': 0.303, 'stop': 0.303, 'jump': 0.394}
```

```
{'move': 0.303, 'stop': 0.303, 'jump': 0.394}
{'move': 0.303, 'stop': 0.303, 'jump': 0.394}
{'move': 0.303, 'stop': 0.303, 'jump': 0.394}
{'move': 0.275, 'stop': 0.275, 'jump': 0.449}
{'move': 0.25, 'stop': 0.25, 'jump': 0.499}
{'move': 0.228, 'stop': 0.228, 'jump': 0.545}
{'move': 0.207, 'stop': 0.207, 'jump': 0.586}
{'move': 0.207, 'stop': 0.207, 'jump': 0.586}
```

**Reflection 2.** In two to three sentences, describe how the action probabilities changed once you flipped the reward, and how this differs from how the DQN in Part 1 learned.

_When the reward was flipped, the action probabilities flipped. Only positively rewarded actions are being boosted. You can see this in the values. This is different than the DQN which is learning the Q values and choosing the action with the highest value instead of adjusting the action probabilities.

## Part 3 — Multi-Agent System

Several agents share one world. Each can **sense**, **decide**, and **act**. They also **communicate** (here, just by seeing where the others are) so they can avoid collisions.

The agent, the world, and the visualization are provided. **Your task** is the coordination rule: an agent should not move forward if another agent is in the cell directly ahead of it.

```
# Cell 10
# Provided: a single agent, the world, and a text visualizer.
class SimpleAgent:
    def __init__(self, world_size):
        self.position = random.randint(0, world_size - 1)
        self.world_size = world_size

    def sense(self, world):
        return world[self.position]

    def decide(self, observation):
        return 'move' if observation == ' ' else 'stop'

    def act(self, action):
        if action == 'move' and self.position < self.world_size - 1:
            self.position += 1
```

```
def create_world(size):
    return [' ' for _ in range(size)]

def visualize(world, positions):
    cells = []
    for i in range(len(world)):
        if i in positions:
            cells.append(str(positions.index(i) + 1))
        else:
            cells.append(world[i])
    return "[" + "][".join(cells) + "]"

print("Helpers ready.")
```

```
Helpers ready.
```

```
# Cell 11
class SimpleMultiAgentSystem:
    def __init__(self, num_agents=3, world_size=10):
        self.agents = [SimpleAgent(world_size) for _ in range(num_agents)]
        self.world_size = world_size
        self.world = create_world(world_size)

    def communicate(self, agent_index):
        # Ready to use. Returns the positions of all the OTHER agents.
        return [a.position for i, a in enumerate(self.agents) if i != agent_index]

    def coordinate_actions(self):
        for i, agent in enumerate(self.agents):
            other_positions = self.communicate(i)
            observation = agent.sense(self.world)

            # TODO: decide the action.

            # If another agent is directly ahead (at agent.position + 1), the action
            # should be 'stop'. Otherwise use agent.decide(observation).

            # Stop if there is an agent directly ahead
            if agent.position + 1 in other_positions:
                action = 'stop'
            else:
                action = agent.decide(observation)
```

```
            # Then call agent.act(action).
            agent.act(action)
```

```
# Cell 12
# Run this to test (provided)
system = SimpleMultiAgentSystem(num_agents=3)
for step in range(5):
    system.coordinate_actions()
    positions = [a.position for a in system.agents]
    print(f"Step {step+1}: {visualize(system.world, positions)}")
```

```
Step 1: [ ][ ][ ][ ][ ][3][1][ ][2][ ]
Step 2: [ ][ ][ ][ ][ ][ ][3][1][ ][2]
Step 3: [ ][ ][ ][ ][ ][ ][ ][3][1][2]
Step 4: [ ][ ][ ][ ][ ][ ][ ][3][1][2]
Step 5: [ ][ ][ ][ ][ ][ ][ ][3][1][2]
```

```
# Cell 13
# EXPERIMENT (TODO)
# Run the same simulation again with num_agents = 5 and watch how often agents stop.

# TODO: write your experiment here.
```

```
# Cell 14
class SimpleMultiAgentSystem:
    def __init__(self, num_agents=5, world_size=10):
        self.agents = [SimpleAgent(world_size) for _ in range(num_agents)]
        self.world_size = world_size
        self.world = create_world(world_size)

    def communicate(self, agent_index):
        # Ready to use. Returns the positions of all the OTHER agents.
        return [a.position for i, a in enumerate(self.agents) if i != agent_index]

    def coordinate_actions(self):
        for i, agent in enumerate(self.agents):
            other_positions = self.communicate(i)
            observation = agent.sense(self.world)

            # TODO: decide the action.
```

```
            # If another agent is directly ahead (at agent.position + 1), the action
            # should be 'stop'. Otherwise use agent.decide(observation).

            # Stop if there is an agent directly ahead
            if agent.position + 1 in other_positions:
                action = 'stop'
            else:
                action = agent.decide(observation)

            # Then call agent.act(action).
            agent.act(action)
```

```
# Cell 15
# Run this to test (provided)
system = SimpleMultiAgentSystem(num_agents=5)
for step in range(5):
    system.coordinate_actions()
    positions = [a.position for a in system.agents]
    print(f"Step {step+1}: {visualize(system.world, positions)}")
```

```
Step 1: [5][2][ ][4][1][ ][3][ ][ ][ ]
Step 2: [ ][5][2][ ][4][1][ ][3][ ][ ]
Step 3: [ ][ ][5][2][ ][4][1][ ][3][ ]
Step 4: [ ][ ][ ][5][2][ ][4][1][ ][3]
Step 5: [ ][ ][ ][ ][5][2][ ][4][1][3]
```

```
# Cell 15.1
# BONUS: Add one obstacle to the multi-agent world

class ObstacleMultiAgentSystem(SimpleMultiAgentSystem):
    def __init__(self, num_agents=5, world_size=10, obstacle_position=None):
        super().__init__(num_agents, world_size)
        # Place a single obstacle in the world
        self.obstacle_position = obstacle_position if obstacle_position is not None else world_size // 2

    def coordinate_actions(self):
        for i, agent in enumerate(self.agents):
            other_positions = self.communicate(i)
            observation = agent.sense(self.world)

            # Stop if another agent is directly ahead
```

```
            if agent.position + 1 in other_positions:
                action = 'stop'
            # Stop if obstacle is directly ahead
            elif agent.position + 1 == self.obstacle_position:
                action = 'stop'
            else:
                action = agent.decide(observation)

            agent.act(action)

# Test run
system = ObstacleMultiAgentSystem(num_agents=3, world_size=10, obstacle_position=5)
for step in range(10):
    system.coordinate_actions()
    positions = [a.position for a in system.agents]
    print(f"Step {step}: {positions} (Obstacle at {system.obstacle_position})")
```

```
Step 0: [1, 7, 9] (Obstacle at 5)
Step 1: [2, 8, 9] (Obstacle at 5)
Step 2: [3, 8, 9] (Obstacle at 5)
Step 3: [4, 8, 9] (Obstacle at 5)
Step 4: [4, 8, 9] (Obstacle at 5)
Step 5: [4, 8, 9] (Obstacle at 5)
Step 6: [4, 8, 9] (Obstacle at 5)
Step 7: [4, 8, 9] (Obstacle at 5)
Step 8: [4, 8, 9] (Obstacle at 5)
Step 9: [4, 8, 9] (Obstacle at 5)
```

**Reflection 3.** In two to three sentences, describe what the coordination rule does when agents get close, and what changed when you used five agents instead of three.

_The coordination rule prevents an agent from moving forward if there is another agent directly ahead of it preventing agents from colliding or trying to occupy the same position. With 3 agents randomly spaced, they moved until they were blocked and with 5 agents, they began clustered so several had to stop and wait for spaces to open up before they could advance. Adding agents increases congestion -- similar to highways or network systems.

## Part 4 — Federated Learning

Several agents each learn on their own, then **share knowledge by averaging** their Q-tables, without ever sharing raw experience. After aggregation, every agent holds the same averaged values.

**Your task:** implement `aggregate_knowledge` to average the Q-values across all agents and write the averaged table back into each one. (Reuses `SimpleDQN` from Part 1.)

```
# Cell 16
class SimpleFederatedSystem:
    def __init__(self, num_agents=3):
        self.agents = [SimpleDQN() for _ in range(num_agents)]

    def aggregate_knowledge(self):
        # TODO:
        # 1. Collect every (state, action) key that appears in ANY agent's q_table.
        all_keys = set()
        for agent in self.agents:
            all_keys.update(agent.q_table.keys())
        # 2. For each key, average that value across all agents (use 0.0 if an agent
        averaged_table = {}
        for key in all_keys:
            values = [agent.q_table.get(key, 0.0) for agent in self.agents]
            averaged_value = sum(values) / len(values)
        #    has not seen it).
        # 3. Replace every agent's q_table with a copy of the averaged table.
            averaged_table[key] = averaged_value
        for agent in self.agents:
            agent.q_table = averaged_table.copy()
```

```
# Cell 17
# Run this to test (provided)
fed = SimpleFederatedSystem(num_agents=3)
for i, agent in enumerate(fed.agents):
    agent.q_table[(f"position_{i}", "move")] = round(random.random(), 3)

print("Before aggregation:")
for i, agent in enumerate(fed.agents):
    print(f"  Agent {i}: {agent.q_table}")

fed.aggregate_knowledge()

print("\nAfter aggregation:")
for i, agent in enumerate(fed.agents):
    print(f"  Agent {i}: {agent.q_table}")
```

```
Before aggregation:
  Agent 0: {('position_0', 'move'): 0.923}
  Agent 1: {('position_1', 'move'): 0.501}
  Agent 2: {('position_2', 'move'): 0.293}

After aggregation:
  Agent 0: {('position_0', 'move'): 0.3076666666666667, ('position_2', 'move'): 0.09766666666666667, ('position_1', 'move'): 0.16
  Agent 1: {('position_0', 'move'): 0.3076666666666667, ('position_2', 'move'): 0.09766666666666667, ('position_1', 'move'): 0.16
  Agent 2: {('position_0', 'move'): 0.3076666666666667, ('position_2', 'move'): 0.09766666666666667, ('position_1', 'move'): 0.16
```

**Reflection 4.** In two to three sentences, explain what aggregation did to each agent's values and give one real situation where sharing averaged knowledge, instead of raw data, would be useful.

_Before the aggregation each agent had its own Q-table. After aggregation, the agents shared an averaged Q-table. Each position took the average of the position divided by the 3 agents. A real-world example is in clinical trials, where scientists share averaged results across multiple experiments instead of raw patient data to protect patient privacy whil still improving collective knowledge.

## Part 5 — Compare Learning Approaches

Finally, run both kinds of agent over many episodes and compare their average reward. The loop is provided, but **you must call the function** at the bottom, since the original code defined it and never ran it.

```
# Cell 18
def compare_learning_approaches(episodes=100):
```

```
    dqn_agent = SimpleDQN()
    policy_agent = SimplePolicyAgent()
    dqn_rewards, policy_rewards = [], []

    for episode in range(episodes):
        # DQN: pick the action with the highest current Q-value
        state, dqn_total = "start", 0
        for _ in range(5):
            action = max(['move', 'stop'], key=lambda a: dqn_agent.get_q_value(state, a))
            reward = random.choice([-1, 1])
            next_state = f"state_{random.randint(1, 5)}"
            dqn_agent.update(state, action, reward, next_state)
            dqn_total += reward
            state = next_state

        # Policy gradient
        policy_total = 0
        for _ in range(5):
            action = policy_agent.choose_action()
            reward = random.choice([-1, 1])
            policy_agent.update_policy(action, reward)
            policy_total += reward

        dqn_rewards.append(dqn_total)
        policy_rewards.append(policy_total)

        if episode % 10 == 0:
            print(f"Episode {episode:>3} | "
                  f"DQN avg {sum(dqn_rewards[-10:]) / 10:+.1f} | "
                  f"Policy avg {sum(policy_rewards[-10:]) / 10:+.1f}")

    return dqn_rewards, policy_rewards
```

```
# Cell 19
# TODO: call compare_learning_approaches() and run it.
compare_learning_approaches()
```

```
    -1,
    1,
    -1,
```

**Reflection 5.** In two to three sentences, state which approach showed steadier average reward in your run and offer one reason why. (The reward here is random, so look at the trend, not a single number.)

_The DQN agent average reward fluctuates, sometimes negative and sometimes positive, showing inconsistency but also that it can improve when it finds a good Q-value. The Policy agent generally hovers around zero or negative but it does occasionally spike. Since its actions are chosen based on probabilities it will be slower stabilizing and will be more variable over time. _

## Final Reflection and Submission

**Final reflection (150 to 250 words).** Which of the five concepts was clearest to you, which was hardest, and what is one real situation where a multi-agent or federated approach would beat a single agent? Reference specific output you saw.

_Of the five concepts, Federated averaging was the clearest to me. I used aggregate_knowledge so that each agent's Q-table was averaged and redistributed. The output before aggregation showed each agent had unique values but after aggregation all the agents had the same position value. The knowledge sharing was pretty easy - since they don't have to rely on only their result but can use their shared average result it is efficient in the case of clinical trials it also anonymizes patient data.

The hardest concept was comparing learning approaches in Part 5. Since the rewards were random, the results were fluctuating and it was more difficult to try to see a trend. For example, DQN sometimes spiked positively (episode 20: +1.6) and then negatively but the Policy agent hovered near zero with occasional spikes. DQN was more volatile while the Policy agent was fairly stable.

Medical research is far more efficient when they can provide averaged results. So, if each agent/medical researcher I used learned locally, then it can contribute to the global model without exposing sensitive data. My medical research average will be used in the collective knowledge while maintaining patient privacy. _

---

### Before you submit

- ☐ Every `# TODO` is implemented and no cell raises `NotImplementedError`.
- ☐ All cells run top to bottom with outputs visible.
- ☐ Both experiment cells (Parts 1, 2, 3, 5) contain your changes and their output.
- ☐ All six reflections are answered in your own words.
- ☐ Export to PDF with **File > Print > Save as PDF** and submit on Canvas.

```
Start coding or generate with AI.
```